Table des matières

Labo 03 — RBAC et gouvernance, de vos yeux 1
Prérequis 1
Étape 0 — Poser le contexte et une table de démo 2
Étape 1 — Les rôles système et l'héritage 2
Étape 2 — RBAC : créer un rôle métier et l'outiller 2
Étape 3 — Masquer une colonne (masking policy) 3
Étape 4 — Filtrer des lignes (row access policy) 4
Étape 5 — Taguer une colonne sensible 4
Étape 6 — Un resource monitor (garde-fou des coûts) 5
Nettoyage 5
Récapitulatif 6

Labo 03 — RBAC et gouvernance, de vos yeux

Module 3/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

But du labo — *Voir fonctionner* le domaine 2 : lire les rôles système et leur héritage ; créer un rôle métier, l'outiller et le rattacher à SYSADMIN ; puis masquer une colonne et filtrer des lignes selon le rôle, taguer une colonne sensible, et poser un resource monitor. Comptez 30 à 40 minutes. Tout se fait dans un worksheet Snowsight.

Prérequis

- Le Cours 00 est fait : entrepôt `SNOWPRO_WH` + base `SNOWPRO_DB`, schéma `SANDBOX`.
- Votre compte peut prendre le rôle `ACCOUNTADMIN` (c'est votre compte d'essai, vous l'avez).
- Édition. Le masquage, les politiques de ligne et les tags exigent Enterprise ou mieux (Cours 01, section 2). C'est l'édition que le Labo 00 fait choisir à l'inscription. Si vous avez pris Standard, les étapes 3 à 5 seront à lire plutôt qu'à faire : l'édition ne se change pas après coup, il faut ouvrir un autre essai.

Un mot sur ACCOUNTADMIN — Pour les manips de gouvernance et le resource monitor, on chausse ACCOUNTADMIN, le rôle qui a déjà tout. En production, on déléguerait ces droits à un rôle de gouvernance dédié, par les privilèges de compte que la théorie liste à la fin de sa section 3 ; sur un compte d'essai solo, ACCOUNTADMIN est le plus direct.

© 2026 iAlexMG. Tous droits réservés.

Étape 0 — Poser le contexte et une table de démo

```
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;

CREATE OR REPLACE TABLE employes (
  id INT, nom STRING, region STRING, courriel STRING, salaire NUMBER
);
INSERT INTO employes VALUES
  (1, 'Alix', 'Québec',  'alix@boite.ca', 72000),
  (2, 'Bo',   'Ontario', 'bo@boite.ca',   68000),
  (3, 'Cam',  'Québec',  'cam@boite.ca',  81000);

SELECT * FROM employes;
```

Ce que vous observez. Trois lignes en clair. C'est notre cobaye : on va progressivement en restreindre la vue selon le rôle.

Étape 1 — Les rôles système et l'héritage

Théorie : `Theorie_03_SNOW.md`, section 2.

```
USE ROLE ACCOUNTADMIN;          -- le rôle qui voit tout
SHOW ROLES;                     -- les rôles système + les vôtres
SELECT CURRENT_ROLE();          -- votre rôle actif

-- L'héritage, en deux coups d'œil :
SHOW GRANTS TO ROLE ACCOUNTADMIN;    -- ACCOUNTADMIN s'est vu accorder SECURITYADMIN et SYSADMIN
SHOW GRANTS TO ROLE SECURITYADMIN;   -- ... et SECURITYADMIN, lui, a reçu USERADMIN
```

Ce que vous observez. Dans `SHOW GRANTS TO ROLE ACCOUNTADMIN`, les lignes `granted_on = ROLE` montrent SECURITYADMIN et SYSADMIN : c'est l'héritage. ACCOUNTADMIN peut donc tout ce qu'ils font. Même logique pour SECURITYADMIN → USERADMIN. La hiérarchie du cours, en vrai.

Étape 2 — RBAC : créer un rôle métier et l'outiller

Théorie : `Theorie_03_SNOW.md`, sections 1, 2 et 3.

```
-- 1) Le BON rôle pour créer un rôle, c'est USERADMIN.
USE ROLE USERADMIN;
CREATE ROLE IF NOT EXISTS analyste_regional;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- 2) On l'outille : USAGE sur les CONTENEURS, puis SELECT sur la table.
USE ROLE ACCOUNTADMIN;
GRANT USAGE  ON WAREHOUSE SNOWPRO_WH                  TO ROLE analyste_regional;
GRANT USAGE  ON DATABASE  SNOWPRO_DB                  TO ROLE analyste_regional;
GRANT USAGE  ON SCHEMA    SNOWPRO_DB.SANDBOX          TO ROLE analyste_regional;
GRANT SELECT ON TABLE     SNOWPRO_DB.SANDBOX.employes TO ROLE analyste_regional;

-- 3) Bonne pratique : rattacher le rôle métier à SYSADMIN.
GRANT ROLE analyste_regional TO ROLE SYSADMIN;

-- 4) Se l'accorder à soi-même pour pouvoir l'endosser.
SET moi = CURRENT_USER();
GRANT ROLE analyste_regional TO USER IDENTIFIER($moi);
--    (Si votre client refuse la variable : remplacez IDENTIFIER($moi) par votre nom,
--     que SELECT CURRENT_USER(); affiche, entre guillemets DOUBLES : c'est un
--     identifiant, pas une chaîne.)

-- 5) Endosser le rôle et vérifier l'accès.
USE ROLE analyste_regional;
USE WAREHOUSE SNOWPRO_WH;
SELECT * FROM SNOWPRO_DB.SANDBOX.employes;   -- OK : le rôle a bien le SELECT
```

Ce que vous observez. Le `SELECT` final passe : `analyste_regional` a reçu, dans l'ordre, le droit de traverser la base et le schéma (`USAGE`) puis de lire la table (`SELECT`). Retirez mentalement le `USAGE` sur le schéma, et l'accès s'effondre : c'est la chaîne des conteneurs du cours (section 3).

Étape 3 — Masquer une colonne (masking policy)

Théorie : `Theorie_03_SNOW.md`, section 5.

```sql
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE MASKING POLICY masque_courriel AS (val STRING) RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN') THEN val
       ELSE '***masqué***' END;

ALTER TABLE SNOWPRO_DB.SANDBOX.employes
  MODIFY COLUMN courriel SET MASKING POLICY masque_courriel;

-- Vu par ACCOUNTADMIN : en clair.
SELECT nom, courriel FROM SNOWPRO_DB.SANDBOX.employes;

-- Vu par le rôle métier : masqué.
```


© 2026 iAlexMG. Tous droits réservés.

```
USE ROLE analyste_regional;
SELECT nom, courriel FROM SNOWPRO_DB.SANDBOX.employes;
```

Ce que vous observez. Même table, même requête, deux résultats : ACCOUNTADMIN lit les vrais courriels, `analyste_regional` voit `*masqué*`. La colonne existe toujours, c'est sa valeur qui est brouillée à la lecture, selon `CURRENT_ROLE()`.

Étape 4 — Filtrer des lignes (row access policy)

Théorie : `Theorie_03_SNOW.md`, section 5.

```
USE ROLE ACCOUNTADMIN;

-- ACCOUNTADMIN voit tout ; les autres, seulement la région « Québec » (démo simplifiée).
CREATE OR REPLACE ROW ACCESS POLICY filtre_region AS (region STRING) RETURNS BOOLEAN ->
  CURRENT_ROLE() = 'ACCOUNTADMIN' OR region = 'Québec';

ALTER TABLE SNOWPRO_DB.SANDBOX.employes
  ADD ROW ACCESS POLICY filtre_region ON (region);

-- ACCOUNTADMIN : les 3 lignes.
SELECT nom, region FROM SNOWPRO_DB.SANDBOX.employes;

-- analyste_regional : seulement le Québec.
USE ROLE analyste_regional;
SELECT nom, region FROM SNOWPRO_DB.SANDBOX.employes;
```

Ce que vous observez. ACCOUNTADMIN voit 3 lignes ; `analyste_regional` n'en voit que 2 (Bo, d'Ontario, disparaît). Contraste avec l'étape 3 : le masquage cache une valeur ; la politique de ligne fait disparaître des lignes entières. Colonne ⇒ masque ; ligne ⇒ filtre.

Étape 5 — Taguer une colonne sensible

Théorie : `Theorie_03_SNOW.md`, section 5.

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE TAG sensibilite;
ALTER TABLE SNOWPRO_DB.SANDBOX.employes
  MODIFY COLUMN salaire SET TAG sensibilite = 'confidentiel';

-- Relire le tag posé :
SELECT SYSTEM$GET_TAG('SNOWPRO_DB.SANDBOX.sensibilite',
                      'SNOWPRO_DB.SANDBOX.employes.salaire', 'COLUMN');
```


© 2026 iAlexMG. Tous droits réservés.

Ce que vous observez. Le tag `sensibilite = 'confidentiel'` est collé à la colonne `salaire`. Seul, un tag ne masque rien : c'est une étiquette. Sa force, c'est qu'on peut y accrocher une masking policy (masquage par tag) pour masquer d'un coup toutes les colonnes ainsi étiquetées, présentes et futures.

Étape 6 — Un resource monitor (garde-fou des coûts)

Théorie : `Theorie_03_SNOW.md`, section 7.

```
USE ROLE ACCOUNTADMIN;

-- Quota volontairement petit + NOTIFY seulement : on POSE le garde-fou sans risquer de couper le labo.
CREATE OR REPLACE RESOURCE MONITOR garde_fou WITH
  CREDIT_QUOTA = 5 FREQUENCY = MONTHLY START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 75  PERCENT DO NOTIFY
           ON 100 PERCENT DO NOTIFY;

SHOW RESOURCE MONITORS;
-- Pour l'attacher à un entrepôt (en prod, on mettrait plutôt SUSPEND) :
--   ALTER WAREHOUSE SNOWPRO_WH SET RESOURCE_MONITOR = garde_fou;
```

Ce que vous observez. `SHOW RESOURCE MONITORS` liste `garde_fou`, son quota (5 crédits), sa fréquence (mensuelle) et sa consommation. On l'a laissé en `NOTIFY` exprès : il alerte sans rien suspendre. En production, un déclencheur `SUSPEND` (ou `SUSPEND_IMMEDIATE`) couperait l'entrepôt une fois le quota atteint.

Nettoyage

```
USE ROLE ACCOUNTADMIN;
-- Détacher les politiques AVANT de les supprimer.
ALTER TABLE SNOWPRO_DB.SANDBOX.employes MODIFY COLUMN courriel UNSET MASKING POLICY;
ALTER TABLE SNOWPRO_DB.SANDBOX.employes DROP ROW ACCESS POLICY filtre_region;
ALTER TABLE SNOWPRO_DB.SANDBOX.employes MODIFY COLUMN salaire UNSET TAG sensibilite;

DROP MASKING POLICY    IF EXISTS masque_courriel;
DROP ROW ACCESS POLICY IF EXISTS filtre_region;
DROP TAG               IF EXISTS sensibilite;
DROP RESOURCE MONITOR  IF EXISTS garde_fou;
DROP TABLE             IF EXISTS SNOWPRO_DB.SANDBOX.employes;

-- Retirer le rôle de démo.
USE ROLE USERADMIN;
DROP ROLE IF EXISTS analyste_regional;
```


© 2026 iAlexMG. Tous droits réservés.

Récapitulatif

- ☐ J’ai lu les rôles système et repéré l’héritage (ACCOUNTADMIN ⊃ SECURITYADMIN / SYSADMIN).
- ☐ J’ai créé un rôle (USERADMIN), l’ai outillé (USAGE + SELECT) et rattaché à SYSADMIN, puis endossé.
- ☐ J’ai masqué une colonne selon le rôle (*masking policy*).
- ☐ J’ai filtré des lignes selon le rôle (*row access policy*).
- ☐ J’ai posé un tag sur une colonne sensible.
- ☐ J’ai créé un resource monitor (quota + déclencheurs + actions).

Tout coché, vous tenez les réflexes du domaine 2 : qui a le droit (RBAC), ce qu’on montre (gouvernance) et le garde-fou des coûts. Cap sur le Cours 04, chargement, déchargement et connectivité.

© 2026 iAlexMG. Tous droits réservés.